

Explications détaillées du code

Projet E2 Nova Studio - fichiers `index.html`, `css/styles.css` et `js/main.js`

Document généré pour comprendre la structure de la page d'accueil, son style visuel et les interactions JavaScript.

Remarque sur le nom du fichier CSS : la demande mentionne `style.css`, mais le fichier réellement utilisé par la page est `css/styles.css`, appelé à la ligne 11 de `index.html`.

Vue d'ensemble

`index.html` contient le contenu de la page : métadonnées, en-tête, navigation, sections visibles, cartes de services et pied de page.

`css/styles.css` définit l'identité visuelle : couleurs, grille, typographie, cartes, boutons, formulaire, animations visuelles de fond et responsive design.

`js/main.js` ajoute le comportement : menu mobile, soumission du formulaire de contact, recentrage des ancres et animation de particules dans le canvas.

index.html

Ce fichier est la page d'accueil. Il ne contient pas de logique complexe : il assemble les éléments sémantiques que le CSS habille et que le JavaScript rend interactifs.

Lignes	Explication
1	<code><!DOCTYPE html></code> indique au navigateur qu'il doit interpréter le document en HTML5.
2	<code><html lang="fr"></code> ouvre le document et précise que la langue principale est le français. C'est utile pour l'accessibilité, le référencement et les lecteurs d'écran.
3-13	Le bloc <code><head></code> contient les informations techniques : encodage, adaptation mobile, description SEO, titre de l'onglet, feuille de style et script JavaScript.
4	<code>UTF-8</code> permet d'afficher correctement les accents, le symbole © et les caractères comme •, ☿, ► ou ♦.
5	La balise <code>viewport</code> rend le site responsive : la largeur de la page suit la largeur réelle de l'écran, notamment sur mobile.
6-9	La <code>meta description</code> résume l'activité d'E2 Nova Studio. Elle peut apparaître dans les résultats de recherche.
10	Le <code><title></code> nomme la page dans l'onglet du navigateur et donne aux moteurs de recherche le sujet principal.
11	Le lien CSS charge <code>css/styles.css</code> . Sans cette ligne, la page garderait une apparence HTML brute.
12	Le script <code>js/main.js</code> est chargé avec <code>defer</code> , ce qui attend que le HTML soit analysé avant d'exécuter le JavaScript.
14-15	Le <code><body></code> commence le contenu visible. Le <code>canvas</code> sert de surface de dessin pour l'animation de particules, tout en étant ignoré par les technologies d'assistance grâce à <code>aria-hidden</code> .
16-32	L'en-tête <code>site-header</code> regroupe le logo, le bouton du menu mobile et la navigation principale.
17-19	Le logo est un lien vers l'accueil. <code>aria-label</code> décrit l'action du lien, et <code>alt</code> décrit l'image.
20-24	Le bouton <code>nav-toggle</code> est le menu hamburger mobile. Ses trois <code>span</code> dessinent les trois barres. <code>aria-controls</code> relie le bouton à la navigation, et <code>aria-expanded</code> indique si le menu est ouvert.
25-31	La navigation contient les liens internes du site. La classe <code>active</code> marque visuellement la page courante.
34-48	Le contenu principal commence avec la section <code>hero</code> , première zone visible de la page. Elle présente la marque, la proposition de valeur et deux appels à l'action.
37	Le paragraphe <code>eyebrow home-eyebrow</code> sert de sur-titre compact. Le <code>
</code> force un retour à la ligne entre les activités.

Lignes	Explication
38	Le <code>h1</code> est le titre principal de la page. La classe <code>logo-gradient</code> applique le dégradé de texte défini dans le CSS.
39-42	Le paragraphe <code>hero-lead</code> explique l'offre en une phrase plus descriptive.
43-46	Les liens d'action sont stylés comme des boutons : l'un mène au contact, l'autre vers les services.
50-61	La section <code>intro-grid</code> présente l'approche. Elle est organisée en deux colonnes sur grand écran grâce au CSS.
63-91	La section <code>service-preview</code> affiche trois cartes de services. Chaque carte est un <code>article</code> , ce qui indique un bloc de contenu autonome.
64-72	Première carte : sites web et outils IA. Le lien <code>#web-ia</code> cible une section précise de <code>services.html</code> .
73-81	Deuxième carte : montage vidéo. Le symbole ▶ sert d'icône visuelle légère.
82-90	Troisième carte : graphisme. Le symbole ✦ renforce l'idée de création visuelle.
93-97	La section <code>cta-band</code> est un appel à l'action final avant le pied de page.
100-103	Le <code>footer</code> contient le copyright et un lien vers les mentions légales.
104-105	Fermeture du <code>body</code> puis du document HTML.

css/styles.css

Le CSS agit comme un système visuel global. Plusieurs règles concernent aussi les autres pages du site, pas seulement `index.html`, par exemple les blocs de contact, les pages de services ou les mentions légales.

Fondations globales

Lignes	Explication
1-17	<code>:root</code> déclare des variables CSS globales : couleurs, ombres, rayon des bordures et largeur maximale. Cela évite de répéter les mêmes valeurs partout.
2-13	Les variables de couleur définissent l'univers visuel : fond sombre, panneaux bleutés, texte clair, bleu, menthe et violet.
14-16	<code>--shadow</code> , <code>--radius</code> et <code>--max</code> standardisent les ombres, arrondis et largeurs de page.
19-21	<code>box-sizing: border-box</code> rend les dimensions plus prévisibles : padding et bordures sont inclus dans la largeur totale.
22-26	Les images et SVG ne dépassent jamais leur conteneur, et gardent leur ratio avec <code>height: auto</code> .
28-31	Le défilement vers les ancres devient fluide, et le débordement horizontal est masqué.
33-44	Le <code>body</code> reçoit la police principale, la couleur de texte, la hauteur minimale et un fond composé de plusieurs gradients. Ces gradients créent l'ambiance spatiale et technologique.
46-57	<code>body::before</code> ajoute une grille lumineuse en arrière-plan. Le pseudo-élément est fixe, non cliquable et placé derrière le contenu.
59-62	Les liens héritent de la couleur du contexte et perdent leur soulignement par défaut.
64-75	Les paragraphes reçoivent une couleur secondaire et un interligne confortable. Les titres, paragraphes et listes peuvent couper les mots longs pour éviter les débordements.
77-85	<code>.nova-canvas</code> positionne le canvas d'animation en plein écran, derrière le site, sans bloquer les clics.

En-tête et navigation

87-104	<code>.site-header</code> crée une barre de navigation collante : elle reste en haut de l'écran, centrée, avec fond transparent flouté, bordure et ombre.
106-112	<code>.brand</code> aligne le logo dans l'en-tête et évite qu'il force une largeur trop grande.
114-120	L'image du logo est affichée en bloc, avec hauteur fixe, ratio conservé et halo bleu léger.
122-126	<code>.main-nav</code> place les liens horizontalement avec un petit espacement.
128-135	Chaque lien de navigation devient une pastille : padding, arrondi complet, couleur discrète et transition douce.
137-142	Au survol ou sur la page active, le lien devient plus clair et reçoit un fond bleu très transparent.
144-152	<code>.nav-toggle</code> est caché par défaut sur desktop. Il devient visible dans les media queries mobiles.
154-161	Les <code>span</code> du bouton dessinent les trois barres du menu hamburger.

Structure de page et hero

163-166	<code>.section</code> centre les sections et limite leur largeur avec <code>--max</code> , tout en gardant une marge latérale sur petits écrans.
168-178	<code>.hero</code> est une grille à deux colonnes : contenu à gauche, espace visuel à droite. <code>isolation</code> aide à gérer les calques des pseudo-éléments.
180-203	<code>.hero::before</code> ajoute l'image <code>assets/ordi.png</code> en arrière-plan, avec gradients et masques pour l'intégrer au fond sombre.
205-214	<code>.hero::after</code> ajoute des halos bleus et violets derrière l'image pour renforcer la profondeur visuelle.
216-231	Ces règles placent certains blocs en contexte relatif et empêchent les contenus de forcer une largeur minimale trop grande dans les grilles.
233-244	<code>.eyebrow</code> définit les petits sur-titres : texte menthe, gras, espacé et en capitales.
246-278	Les titres reçoivent des marges nulles, une couleur claire et des tailles différentes. Le <code>h1</code> est très grand sur desktop.
260-266	<code>.logo-gradient</code> rend le texte transparent et affiche un dégradé dans les lettres grâce à <code>background-clip: text</code> .
280-285	<code>.hero-lead</code> et les paragraphes de héros de pages internes ont une largeur limitée, une marge supérieure et une taille plus lisible.
287-320	Les boutons sont alignés en flex, arrondis, animés au survol, puis déclinés en bouton principal et bouton secondaire transparent.

Cartes, sections et contenus réutilisables

322-332	<code>.hero-panel</code> prépare un panneau décoratif. Il n'est pas utilisé dans <code>index.html</code> actuel, mais peut servir ailleurs ou dans une version précédente.
334-345	Plusieurs composants partagent le même style de carte : bordure fine, fond semi-transparent, ombre et flou de fond.
347-390	Les règles <code>.orbit-card</code> , <code>.service-symbol</code> et variantes stylent des cartes décoratives ou de services utilisées sur d'autres pages.
392-403	<code>.intro-grid</code> crée la section d'introduction en deux colonnes, avec espacement vertical et bordure haute.
405-418	<code>.service-preview</code> et <code>.values-grid</code> affichent trois colonnes de cartes. Les cartes ont une hauteur minimale, du padding et un arrondi cohérent.
420-430	<code>.service-icon</code> dessine le carré qui contient l'icône de chaque carte de service.
432-442	Ces lignes ajustent l'espacement des textes de cartes et rendent les liens d'exploration visibles en couleur menthe.
444-458	<code>.cta-band</code> crée le bandeau d'appel à l'action : bordure bleue, fond dégradé transparent, ombre et espace intérieur.

Pages internes, formulaire et pied de page

460-488	Ces règles concernent les pages internes : héros de page, section en deux colonnes et bloc portrait carré avec image couvrante.
490-529	<code>.content-stack</code> et <code>.stat-row</code> organisent les textes et statistiques, probablement sur la page À propos.
531-577	<code>.services-detail</code> et <code>.detail-card</code> structurent les fiches détaillées de services : numéro, texte, liste sans puces natives et puce personnalisée menthe.
579-597	<code>.contact-layout</code> , <code>.contact-form</code> et les éléments associés définissent la largeur, l'espacement et l'apparence de la zone de contact.
599-627	Ces règles gèrent les lignes de formulaire et la case RGPD : alignement horizontal, taille de case stable et couleur d'accent.
629-663	Les champs <code>input</code> , <code>select</code> et <code>textarea</code> ont une largeur complète, un fond sombre, une bordure, une police héritée et un halo au focus.
665-690	<code>.form-status</code> réserve l'espace du message de retour, tandis que <code>.contact-aside</code> et <code>.contact-chip</code> stylent les informations complémentaires.
692-723	Les règles des mentions légales et du pied de page définissent les cartes de contenu légal et l'alignement du footer.

Responsive design

725-814	À partir de 920 px et moins, le menu mobile apparaît, la navigation devient un panneau vertical masqué par défaut, les grilles passent en une colonne et les tailles de titres diminuent.
735-754	La navigation mobile est en position absolue sous l'en-tête. La classe <code>.open</code> , ajoutée par JavaScript, la rend visible.
789-798	L'image de fond du hero devient plus transparente et mieux positionnée sur écrans moyens.
816-821	Entre 641 px et 920 px, les grilles de services et valeurs passent à deux colonnes au lieu d'une seule.
823-882	À 640 px et moins, les marges latérales diminuent, le logo est réduit, les boutons prennent toute la largeur et les blocs complexes deviennent plus compacts.
885-951	À 430 px et moins, la version très mobile réduit encore l'en-tête, les titres et les espacements. Les lignes 939-950 ajoutent les couleurs de succès/erreur et l'état désactivé du bouton de formulaire.

js/main.js

Le JavaScript est écrit de manière défensive : chaque fonctionnalité vérifie d'abord que les éléments HTML existent. Cela permet au même fichier d'être chargé sur plusieurs pages sans provoquer d'erreurs.

Menu mobile

1-2	Le script récupère le bouton du menu mobile et la navigation principale avec <code>document.querySelector</code> .
4	La condition vérifie que les deux éléments existent avant d'ajouter des comportements.
5-8	<code>closeNav</code> ferme le menu en retirant la classe <code>open</code> et en remettant <code>aria-expanded</code> à <code>false</code> .
10-13	Au clic sur le bouton, <code>classList.toggle("open")</code> ouvre ou ferme le menu. La valeur retournée est convertie en texte pour l'attribut ARIA.
15-17	Chaque lien du menu ferme la navigation au clic. C'est pratique sur mobile après avoir choisi une page.
19-21	Si la fenêtre repasse au-dessus de 920 px, le menu mobile est fermé pour éviter un état incohérent au changement de taille.
23-25	La touche Échap ferme aussi le menu, ce qui améliore l'accessibilité clavier.

Formulaire de contact

28-32	Le script cherche le formulaire, le message de statut et le bouton d'envoi. Sur la page d'accueil, le formulaire n'existe pas, donc ce bloc ne s'exécute pas.
33-37	Lors de l'envoi, <code>event.preventDefault()</code> empêche le rechargement de page. Si le statut ou le bouton manque, la fonction s'arrête.
38-42	Le texte initial du bouton est conservé, le message est vidé, les classes de retour sont retirées, puis le bouton est désactivé avec le texte "Envoi en cours...".
43-49	Le bloc <code>try</code> prépare un <code>FormData</code> et envoie les données à Web3Forms via <code>fetch</code> en méthode POST.
50-54	Si la réponse HTTP échoue ou si Web3Forms renvoie <code>success: false</code> , une erreur est déclenchée avec un message explicite.
55-58	En cas de réussite, un message positif est affiché, la classe <code>is-success</code> est ajoutée et le formulaire est réinitialisé.
59-63	En cas d'erreur, le problème est écrit dans la console et un message d'échec est affiché à l'utilisateur avec la classe <code>is-error</code> .
64-67	Le bloc <code>finally</code> s'exécute dans tous les cas : il réactive le bouton et restaure son texte initial.

Ancres et défilement

71-72	<code>centerHashTarget</code> commence par vérifier si l'URL contient une ancre, par exemple <code>#web-ia</code> .
74	Le script cherche dans le document l'élément correspondant à cette ancre.
76-80	Si l'élément existe, un léger délai laisse la page se charger, puis <code>scrollIntoView</code> le place au centre de l'écran avec un défilement fluide.
83-84	La fonction est appelée au chargement, puis à chaque changement d'ancre dans l'URL.

Animation du canvas

86-94	Le script récupère le canvas, son contexte 2D, initialise la liste de particules, la position du pointeur, les dimensions et l'identifiant d'animation.
96-109	<code>createParticles</code> vide la liste puis crée un nombre de particules adapté à la taille de l'écran, entre 36 et 86.
101-107	Chaque particule reçoit une position aléatoire, une vitesse horizontale/verticale légère et une taille aléatoire.
111-121	<code>resize</code> adapte le canvas à la taille de la fenêtre et au ratio de pixels de l'écran. Le ratio est limité à 2 pour garder de bonnes performances.
123-130	<code>drawLine</code> dessine un segment bleu semi-transparent entre deux points. L'opacité est passée en paramètre.
132-147	<code>render</code> efface le canvas, déplace chaque particule, la fait réapparaître de l'autre côté si elle sort de l'écran, puis la dessine sous forme de cercle.
149-156	Chaque particule est comparée aux suivantes. Si deux particules sont proches de moins de 118 px, une ligne les relie avec une opacité proportionnelle à leur distance.
158-164	Si le pointeur est actif, les particules proches de la souris ou du doigt sont reliées au pointeur jusqu'à 150 px.
167	<code>requestAnimationFrame</code> relance <code>render</code> au prochain rafraîchissement de l'écran. C'est la boucle d'animation.
170-178	Les événements mettent à jour la taille du canvas, la position du pointeur et l'état actif/inactif du pointeur.
180-181	Le canvas est dimensionné une première fois, puis l'animation démarre.
183-189	Quand l'onglet devient masqué, l'animation est annulée pour économiser des ressources. Quand l'onglet redevient visible, elle reprend.

Relations entre les fichiers

- `index.html` crée le canvas ligne 15, `styles.css` le place en arrière-plan lignes 77-85, et `main.js` l'anime lignes 86-190.
- `index.html` crée le bouton mobile lignes 20-24, `styles.css` le dessine lignes 144-161 et l'affiche en mobile lignes 730-733, puis `main.js` le rend interactif lignes 1-26.
- Les liens de services de `index.html` lignes 71, 80 et 89 pointent vers des ancres de `services.html`. `main.js` améliore leur positionnement à l'écran lignes 71-84.
- Les classes comme `button`, `service-card`, `cta-band` et `hero` sont déclarées dans le HTML puis transformées visuellement par le CSS.

Points à surveiller

- Les lignes 322-390 du CSS contiennent des styles de composants qui ne sont pas utilisés directement dans `index.html`, mais peuvent servir aux autres pages.
- Les styles `.form-status.is-success` et `.form-status.is-error` sont actuellement placés dans la media query `max-width: 430px`. Sur desktop, les classes sont ajoutées par JavaScript, mais les couleurs spécifiques de succès/erreur ne s'appliquent pas forcément.
- La ligne 116 du CSS est plus indentée que les autres propriétés du bloc `.brand img`. Ce n'est pas bloquant, mais un formatage homogène améliorerait la lisibilité.